

SOURCES:

SLB.py

```
import argparse, os, random, struct
import numpy as np
from PIL import Image

MAGIC = b"STG1"

def _bits(data: bytes) -> np.ndarray:
    return np.unpackbits(np.frombuffer(data, dtype=np.uint8))

def _bytes(bits: np.ndarray) -> bytes:
    return np.packbits(bits).tobytes()

def _order(total: int, start: int, key):
    """Urutan index byte yang dipakai setelah header (sequential / acak)."""
    idx = list(range(start, total))
    if key:
        random.Random(key).shuffle(idx)
    return idx

def encode(cover, out, text=None, file=None, key=None):
    img = Image.open(cover).convert("RGB")
    flat = np.array(img, dtype=np.uint8).flatten()

    if file:
        name = os.path.basename(file).encode()
        body = open(file, "rb").read()
        kind = 1
    else:
        name, body, kind = b"", text.encode(), 0

    payload = struct.pack(">BB", kind, len(name)) + name + body
    # header: magic + panjang payload (selalu disimpan sekuensial)
    header = MAGIC + struct.pack(">I", len(payload))
    hbits, pbits = _bits(header), _bits(payload)

    if len(hbits) + len(pbits) > flat.size:
        raise ValueError(f"Pesan terlalu besar. Kapasitas: {(flat.size - len(hbits)) // 8} byte")

    flat[:len(hbits)] = (flat[:len(hbits)] & 0xFE) | hbits
    pos = np.array(_order(flat.size, len(hbits), key)[:len(pbits)])
    flat[pos] = (flat[pos] & 0xFE) | pbits

    Image.fromarray(flat.reshape(np.array(img).shape)).save(out, "PNG")
    print(f"[OK] Stego-image disimpan: {out} ({len(payload)} byte disisipkan)")

def decode(stego, key=None, outdir="."):
    flat = np.array(Image.open(stego).convert("RGB"),
dtype=np.uint8).flatten()

    hlen = 8 * 8 # 4 byte magic + 4 byte panjang
    header = _bytes(flat[:hlen] & 1)
    if header[:4] != MAGIC:
```

```
        raise ValueError("Tidak ditemukan pesan tersembunyi.")
    plen = struct.unpack(">I", header[4:])[0]

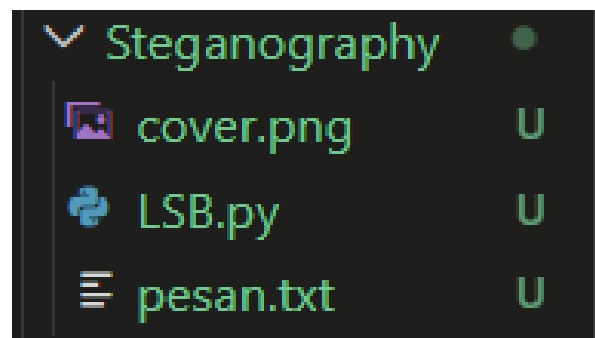
    pos = np.array(_order(flat.size, hlen, key)[:plen * 8])
    payload = _bytes(flat[pos] & 1)

    kind, nlen = struct.unpack(">BB", payload[:2])
    name, body = payload[2:2 + nlen], payload[2 + nlen:]
    if kind == 0:
        print("[OK] Pesan teks:", body.decode(errors="replace"))
    else:
        os.makedirs(outdir, exist_ok=True)
        path = os.path.join(outdir, "extracted_" + name.decode())
        open(path, "wb").write(body)
        print(f"[OK] File diekstrak: {path}")

if __name__ == "__main__":
    p = argparse.ArgumentParser(description="Steganografi LSB")
    p.add_argument("mode", choices=["encode", "decode"])
    p.add_argument("-c", "--cover")
    p.add_argument("-s", "--stego")
    p.add_argument("-t", "--text")
    p.add_argument("-f", "--file")
    p.add_argument("-o", "--out", default=None)
    p.add_argument("-k", "--key", default=None)
    a = p.parse_args()

    if a.mode == "encode":
        if not a.cover or not (a.text or a.file):
            p.error("encode butuh -c dan (-t atau -f)")
        encode(a.cover, a.out or "stego.png", a.text, a.file, a.key)
    else:
        if not a.stego:
            p.error("decode butuh -s")
        decode(a.stego, a.key, a.out or ".")
```

Cover.py



Pesan.txt

```
ini pesannya
```

RESULT

```
PS C:\KULIAH\SEMS 5\PRAK\KRIPTO\240068-Kripto26\Steganography> python LSB.py encode -c cover.png -t "pesan rahasia" -o stego.png
>> python LSB.py decode -s stego.png
[OK] Stego-image disimpan: stego.png (15 byte disisipkan)
[OK] Pesan teks: pesan rahasia
PS C:\KULIAH\SEMS 5\PRAK\KRIPTO\240068-Kripto26\Steganography> python LSB.py decode -s stego.png
[OK] Pesan teks: pesan rahasia
PS C:\KULIAH\SEMS 5\PRAK\KRIPTO\240068-Kripto26\Steganography> python LSB.py encode -c cover.png -t "pesan rahasia" -o stego2.png -k kunci
>> python LSB.py decode -s stego2.png -k kunci
[OK] Stego-image disimpan: stego2.png (15 byte disisipkan)
[OK] Pesan teks: pesan rahasia
PS C:\KULIAH\SEMS 5\PRAK\KRIPTO\240068-Kripto26\Steganography> python LSB.py encode -c cover.png -f pesan.txt -o stego3.png
>> python LSB.py decode -s stego3.png -o hasil
[OK] Stego-image disimpan: stego3.png (23 byte disisipkan)
[OK] File diekstrak: hasil\extracted_pesan.txt
PS C:\KULIAH\SEMS 5\PRAK\KRIPTO\240068-Kripto26\Steganography>
```

extracted_pesan.txt

ini pesannya

Perbandingan gambar:

stegno.png



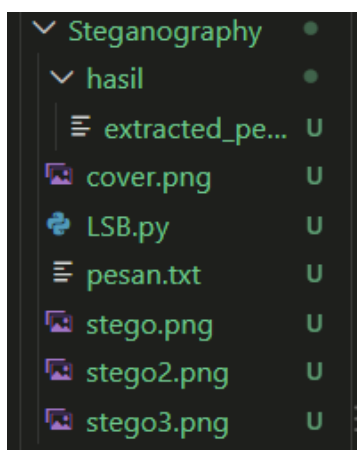
stegno2.png



stegno3.png



Struktur folder akhir:



CARA KERJA

1. Deskripsi Program

Program ini adalah implementasi steganografi dengan metode Least Significant Bit (LSB) menggunakan Python. Program dapat menyembunyikan pesan teks atau file (misalnya gambar atau dokumen teks) ke dalam citra PNG 24-bit, lalu mengekstraknya kembali. Tersedia dua mode penyisipan, yaitu sequential dan acak (menggunakan stego-key).

2. Metode yang Digunakan

Citra true color terdiri dari pixel, dan setiap pixel terdiri dari 3 byte (R, G, B). Pada metode LSB, bit paling kanan dari setiap byte diganti dengan satu bit pesan. Perubahan nilai hanya sebesar 1 (misalnya 202 menjadi 203), sehingga tidak terlihat oleh mata manusia dan stego-image tampak sama dengan citra aslinya.

3. Cara Kerja Program

a. Format data yang disisipkan

Data yang disisipkan terdiri dari header dan payload.

- Header (8 byte): 4 byte penanda STG1 dan 4 byte panjang payload. Header selalu disisipkan secara sekuensial di awal citra agar decoder tahu cara membacanya.
- Payload: 1 byte tipe (0 = teks, 1 = file), 1 byte panjang nama file, nama file, lalu isi pesan.

b. Proses encode

1. Citra cover dibuka dan diubah ke mode RGB, lalu diratakan menjadi array byte.
2. Pesan dibungkus menjadi payload, lalu dibuat header-nya.
3. Header dan payload diubah menjadi deretan bit.
4. Program memeriksa kapasitas. Jika bit pesan melebihi jumlah byte citra, proses dihentikan.
5. Bit header disisipkan ke LSB 64 byte pertama.
6. Urutan byte untuk payload ditentukan. Tanpa key, urutannya berurutan. Dengan key, urutan indeks diacak memakai PRNG dengan key sebagai seed.
7. LSB pada byte-byte tersebut diganti dengan bit payload.
8. Hasilnya disimpan sebagai PNG (lossless) agar bit LSB tidak rusak.

c. Proses decode

1. Stego-image dibuka dan diratakan menjadi array byte.
2. LSB 64 byte pertama dibaca sebagai header. Jika penanda bukan STG1, berarti tidak ada pesan tersembunyi.
3. Panjang payload diambil dari header.
4. Urutan byte dibangkitkan dengan cara yang sama seperti saat encode. Key harus sama persis.
5. LSB pada urutan tersebut dibaca dan disusun kembali menjadi byte payload.
6. Jika tipe teks, pesan ditampilkan. Jika tipe file, file disimpan dengan nama `extracted_<nama_asli>`.

d. Kapasitas

Kapasitas = $(\text{lebar} \times \text{tinggi} \times 3 - 64) / 8$ byte.

4. Cara Menjalankan

```
python LSB.py encode -c cover.png -t "pesan rahasia" -o stego.png
python LSB.py decode -s stego.png
python LSB.py encode -c cover.png -f pesan.txt -o stego3.png -k
kunci
python LSB.py decode -s stego3.png -k kunci -o hasil
```

5. Link Repository GitHub

<https://github.com/nandenandeya/240068-Kripto26.git>